AULA 01
INTRODUÇÃO AO PYTHON

Programação em Python • Parte 1 de 2
Curso de Capacitação em Desenvolvimento Full Stack
Tema: Introdução ao Python • 6 horas
Professor: Msc. Hygo Sousa de Oliveira

Roteiro da aula

O que vamos ver nesta aula

1. História e filosofia do Python
2. Como o Python executa o código
3. Instalação do Python e escolha da IDE
4. Ambiente virtual (venv) e dependências
5. Primeiro script, módulo sys e leitura de erros
6. Primeiro contato com variáveis e tipos

Objetivos de aprendizagem

Ao final da aula você será capaz de:

- **Contar a história e a filosofia do Python**
  Guido van Rossum, o CWI e o Zen of Python
- **Instalar o Python 3.12 e configurar uma IDE**
  VS Code, PyCharm Community ou IDLE
- **Criar e ativar um ambiente virtual**
  venv, pip, requirements.txt e .gitignore
- **Executar scripts e interpretar mensagens de erro**
  print(), módulo sys, traceback e type()

Módulo 1 – Introdução ao Python

- História e filosofia do Python
- Instalação e ambiente de desenvolvimento
- Variáveis, tipos, operadores e expressões
- Entrada e saída de dados
- Comentários e documentação

O que é Python?

Linguagem de alto nível, interpretada e de propósito geral

1 Criada por Guido van Rossum

Idealizada no CWI (Centrum Wiskunde & Informatica), na Holanda.

2 Sucessora da linguagem ABC

Com foco em tratamento de exceções e na interface com o sistema operacional Amoeba.

3 Clareza e legibilidade

Rejeita o código excessivamente verboso: a indentação faz parte da sintaxe.

4 Multiplataforma e "baterias inclusas"

Roda em Windows, Linux e macOS e traz uma vasta biblioteca padrão.

Linha do tempo do Python

Da criação ao Python 3 que usamos hoje

1991

Primeira versão

Guido van Rossum publica o Python, criado no CWI.

2000

Python 2.0

Chegam as compreensões de lista e o coletor de lixo com ciclos.

2008

Python 3.0

Revisão da linguagem: a base do Python moderno.

2020

Fim do suporte

O Python 2 deixa de ser mantido; usamos sempre Python 3.

NOTA O nome vem do grupo de humor britânico **Monty Python's Flying Circus**, e não da cobra.

O Zen of Python

19 princípios para escrever código "pythônico"

Princípio (original)	Tradução	Na prática
Explicit is better than implicit	Explícito é melhor que implícito	Converta tipos com `int()`, sem "mágica"
Simple is better than complex	Simples é melhor que complexo	Prefira a solução mais direta
Readability counts	Legibilidade conta	Nomes claros e indentação consistente
Flat is better than nested	Plano é melhor que aninhado	Evite `if` dentro de `if` dentro de `if`
Errors should never pass silently	Erros nunca devem passar em silêncio	Trate exceções, não as esconda

EM SALA Rode `import this` no terminal para ver os 19 princípios completos.

Como o Python executa seu código

Interpretado, mas com uma etapa intermediária

Código-fonte
arquivo .py

→

Bytecode
formato intermediário

→

PVM
Máquina Virtual Python

→

Resultado
saída no terminal

Ciência de dados
Análise e modelagem de dados.

Automação
Scripts para tarefas repetitivas.

Web e APIs
Sistemas que rodam no servidor.

Ensino
Ótima linguagem para aprender a programar.

NOTA O bytecode garante portabilidade entre plataformas, mas exige atenção ao gerenciamento de memória.

Instalando o Python

Três passos, em qualquer sistema operacional

1. **Baixe o instalador em python.org/downloads**
   Escolha o Python 3 para o seu sistema (neste curso: 3.12 ou superior).
2. **Marque a opção Add Python to PATH (Windows)**
   Permite executar o interpretador pelo terminal.
3. **Conclua a instalação e verifique no terminal**
   Prompt/PowerShell no Windows; Terminal no macOS e Linux.

ATENÇÃO Sem marcar "Add Python to PATH", o comando `python` não será reconhecido no Windows.

Verificando a instalação

O terminal confirma a versão; o REPL executa linha a linha

TERMINAL

```
python --version
python3 --version  # Linux/macOS
```

SAÍDA

```
Python 3.12.x
```

REPL (MODO INTERATIVO)

```
python
>>> 2 + 3
5
>>> print("Olá!")
Olá!
>>> exit()
```

EM SALA O REPL executa uma linha por vez; um arquivo `.py` executa o programa inteiro.

Escolhendo a IDE

Editores simples funcionam, mas IDEs aceleram o desenvolvimento

VS Code

Leve, altamente customizável e com extensões oficiais para Python.

PyCharm Community

Robusta, com suporte nativo a projetos e gerenciamento de dependências.

IDLE

Ambiente padrão que acompanha o instalador do Python; ideal para iniciantes.

NOTA Uma IDE oferece realce de sintaxe, autocompletar e depuração integrada.

Configurando o VS Code

Do download ao primeiro arquivo .py

1. **Instale o VS Code**
   code.visualstudio.com
2. **Instale a extensão Python (Microsoft)**
   Painel Extensions, na barra lateral.
3. **Abra a pasta do projeto**
   Menu File > Open Folder.
4. **Selecione o interpretador**
   Ctrl+Shift+P e escolha "Python: Select Interpreter".
5. **Crie o arquivo hello.py e execute**
   Botão Run no canto superior ou pelo terminal integrado.

Ambiente virtual (venv)

Um ambiente isolado para cada projeto

TERMINAL

```
python -m venv .venv              # cria o ambiente
.venv\Scripts\activate            # ativa (Windows)
source .venv/bin/activate         # ativa (Linux/macOS)
pip install requests              # instala só neste ambiente
pip freeze > requirements.txt     # registra as dependências
deactivate                        # sai do ambiente
```

NOTA Cada projeto tem as próprias versões de pacotes, sem conflitos. No curso usamos o Python 3.12.

.gitignore e requirements.txt

O que vai (e o que não vai) para o GitHub

.gitignore

```
.venv/
__pycache__/
*.pyc
.env
```

requirements.txt

```
requests==2.32.3
python-dotenv==1.0.1
```

- **O .venv/ fica de fora**
  É pesado e específico de cada máquina.
- **O requirements.txt recria o ambiente**
  pip install -r requirements.txt
- **Nunca envie senhas ou chaves**
  Arquivos .env devem estar no .gitignore.

Seu primeiro programa

print() exibe valores na tela

primeiro.py

```
# primeiro.py
print("Olá, mundo!")
print("Meu nome é Ana")
print("Python", 3.12, True)
```

SAÍDA

```
Olá, mundo!
Meu nome é Ana
Python 3.12 True
```

EM SALA Digite junto com a turma, salve como `primeiro.py` e execute `python primeiro.py` no terminal.

O módulo sys

Informações sobre o interpretador e o sistema

sys_info.py

```
import sys
print(sys.version_info.major, sys.version_info.minor)
print(sys.platform)
print(sys.executable)
```

SAÍDA (EXEMPLO – VARIA POR MÁQUINA)

```
3 12
win32
C:\projeto\.venv\Scripts\python.exe
```

DICA `sys.executable` confirma se o venv está ativo; `sys.argv` guarda os argumentos da linha de comando.

Lendo erros: o traceback

A mensagem de erro é uma aliada: leia de baixo para cima

erro.py

```
nome = "Ana"
print("Olá, " + nome)
print("Total: " + total)
```

SAÍDA

```
Olá, Ana
Traceback (most recent call last):
  File "erro.py", line 3, in <module>
    print("Total: " + total)
                      ^^^^^
NameError: name 'total' is not defined
```

EM SALA Leia de baixo para cima: 1) tipo e mensagem do erro, 2) arquivo e linha, 3) trecho apontado pelos acentos circunflexos.

Os erros mais comuns

Reconhecer o nome do erro já aponta o caminho da solução

Erro	Causa típica	Exemplo
`SyntaxError`	Sintaxe inválida	`print("oi"`
`IndentationError`	Recuo incorreto ou ausente	`linha sem recuo após `if` `
`NameError`	Nome não definido	`print(total)`
`TypeError`	Operação entre tipos incompatíveis	`"2" + 3`
`ValueError`	Valor inadequado para o tipo	`int("abc")`
`ZeroDivisionError`	Divisão por zero	`10 / 0`

DICA Os erros "propositais" do repositório treinam exatamente essa leitura.

Roteiro de laboratório

Do zero ao primeiro script: faça junto com a turma

1. **Crie a pasta do projeto**
   `mkdir meu_projeto` e depois `cd meu_projeto`
2. **Crie o ambiente virtual**
   `python -m venv .venv`
3. **Ative o ambiente**
   `.venv\Scripts\activate` (Windows) ou `source .venv/bin/activate`
4. **Crie o arquivo primeiro.py**
   Conteúdo: `print("Olá, Python!")`
5. **Execute o script**
   `python primeiro.py`
6. **Registre as dependências**
   `pip freeze > requirements.txt`

Primeiro contato: variáveis e type()

Dar nome a um valor e descobrir o seu tipo

tipos.py

```
nome = "Ana"
idade = 28
altura = 1.65
estudante = True
print(type(nome), type(idade))
print(type(altura), type(estudante))
```

SAÍDA

```
<class 'str'> <class 'int'>
<class 'float'> <class 'bool'>
```

NOTA Na próxima aula vamos aprofundar tipos primitivos, operadores e conversões.

Atividades da Aula 1

Módulo 1: ambiente, primeiros scripts e tipos

- **Ex. 00 – Configuração do ambiente virtual**
  Python 3.12, `.gitignore` e `requirements.txt`
- **Ex. 01 a 05 – Primeiros passos**
  Primeiro script, módulo `sys`, Zen of Python e leitura de erros propositais
- **Ex. 06 – Variáveis e tipos primitivos**
  Seis variáveis inspecionadas com `type()`

GIT Enunciados, instruções de execução e de entrega:
github.com/Hygo/python-fullstack-iteam-2sem

Obrigado(a)!

Próxima aula: Variáveis, tipos, operadores e entrada/saída
ITEAM – Instituto Tecnológico Educacional da Amazônia